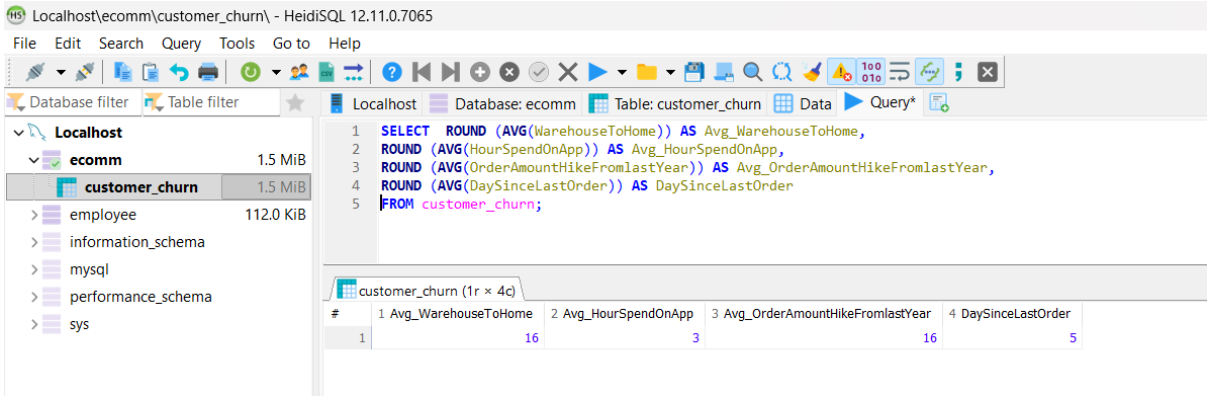
DATA CLEANING

1. SELECT ROUND (AVG(WarehouseToHome)) AS Avg_WarehouseToHome, ROUND (AVG(HourSpendOnApp)) AS Avg_HourSpendOnApp, ROUND (AVG(OrderAmountHikeFromlastYear)) AS Avg_OrderAmountHikeFromlastYear, ROUND (AVG(DaySinceLastOrder)) AS DaySinceLastOrder FROM customer_churn;



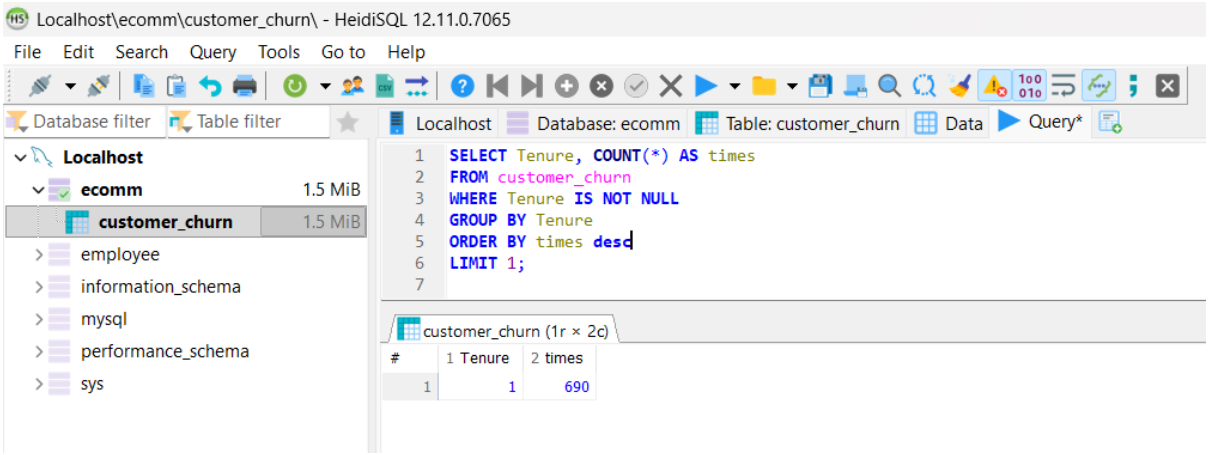
UPDATE customer_churn SET WarehouseToHome = (SELECT AvgValue FROM (SELECT ROUND(AVG(WarehouseToHome)) AS AvgValue FROM customer_churn) AS temp) WHERE WarehouseToHome IS NULL;

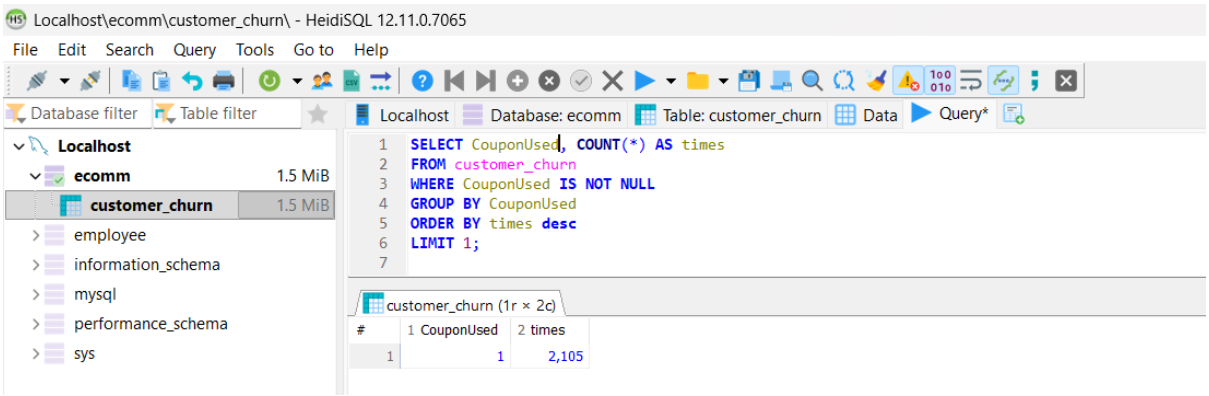
UPDATE customer_churn SET HourSpendOnApp = (SELECT AvgValue FROM (SELECT ROUND(AVG(HourSpendOnApp)) AS AvgValue FROM customer_churn) AS temp) WHERE HourSpendOnApp IS NULL;

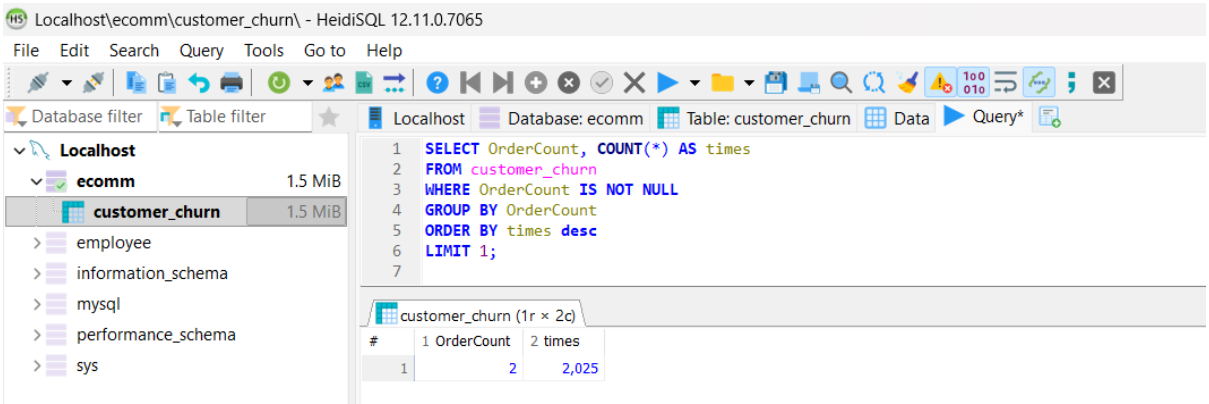
UPDATE customer_churn SET OrderAmountHikeFromlastYear = (SELECT AvgValue FROM (SELECT ROUND(AVG(OrderAmountHikeFromlastYear)) AS AvgValue FROM customer_churn) AS temp) WHERE OrderAmountHikeFromlastYear IS NULL;

UPDATE customer_churn SET DaySinceLastOrder = (SELECT AvgValue FROM (SELECT ROUND(AVG(DaySinceLastOrder)) AS AvgValue FROM customer_churn) AS temp) WHERE DaySinceLastOrder IS NULL;

2. Mode





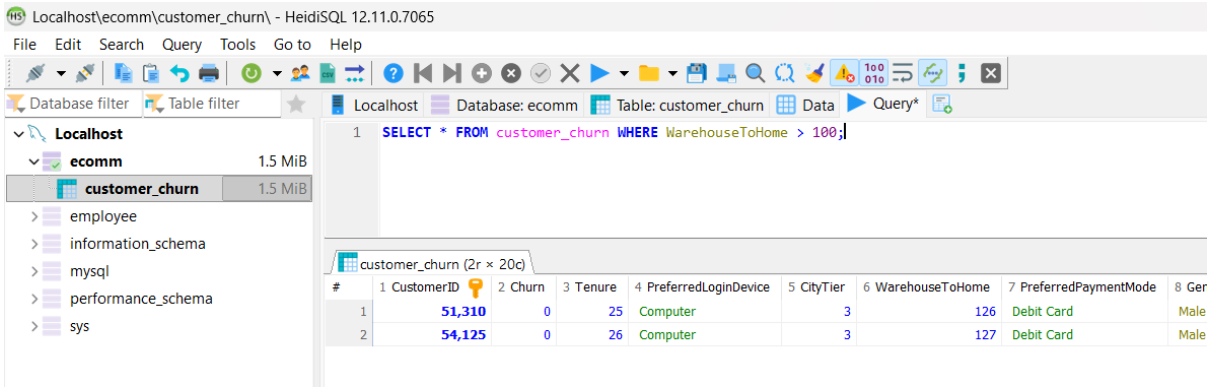


UPDATE customer_churn SET Tenure = (SELECT ModeValue FROM (SELECT Tenure AS ModeValue FROM customer_churn WHERE Tenure IS NOT NULL GROUP BY Tenure ORDER BY COUNT(*) DESC LIMIT 1) AS temp) WHERE Tenure IS NULL;

UPDATE customer_churn SET CouponUsed = (SELECT ModeValue FROM (SELECT CouponUsed AS ModeValue FROM customer_churn WHERE CouponUsed IS NOT NULL GROUP BY CouponUsed ORDER BY COUNT(*) DESC LIMIT 1) AS temp) WHERE CouponUsed IS NULL;

UPDATE customer_churn SET OrderCount = (SELECT ModeValue FROM (SELECT OrderCount AS ModeValue FROM customer_churn WHERE OrderCount IS NOT NULL GROUP BY OrderCount ORDER BY COUNT(*) DESC LIMIT 1) AS temp) WHERE OrderCount IS NULL;

3. SELECT * FROM customer_churn WHERE WarehouseToHome > 100;



DELETE FROM customer_churn WHERE WarehouseToHome > 100;

Dealing with Inconsistencies:

1. Replace Phone, Mobile with MobilePhone

UPDATE customer_churn SET PreferredLoginDevice = 'Mobile Phone' WHERE PreferredLoginDevice = 'Phone';

UPDATE customer_churn SET PreferedOrderCat = 'Mobile Phone' WHERE PreferedOrderCat = 'Mobile';

2. Correct Payment mode

UPDATE customer_churn SET PreferredPaymentMode = 'Cash on Delivery' WHERE PreferredPaymentMode = 'COD';

UPDATE customer_churn SET PreferredPaymentMode = 'Credit Card' WHERE PreferredPaymentMode = 'CC';

Data Transformation :

1. Column Renaming

ALTER TABLE customer_churn RENAME COLUMN PreferedOrderCat TO PreferredOrderCat;

ALTER TABLE customer_churn RENAME COLUMN HourSpendOnApp TO HourSpentOnApp;

2. Creating New Columns

ALTER TABLE customer_churn ADD ComplaintReceived VARCHAR(5);

UPDATE customer_churn SET ComplaintReceived = case when complain = 1 then 'Yes' ELSE 'No' END;

ALTER TABLE customer_churn ADD ChurnStatus VARCHAR(10);

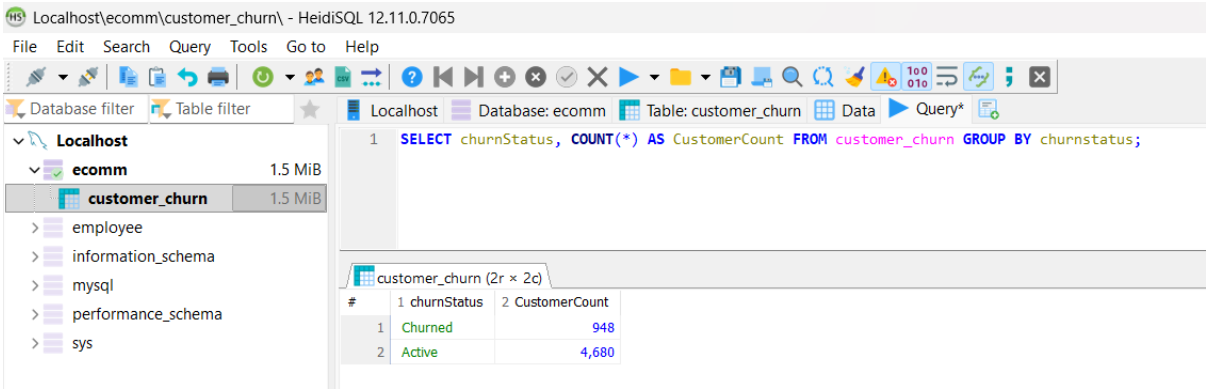
UPDATE customer_churn SET ChurnStatus = CASE WHEN Churn = 1 THEN 'Churned' ELSE 'Active' END;

3. Column Dropping

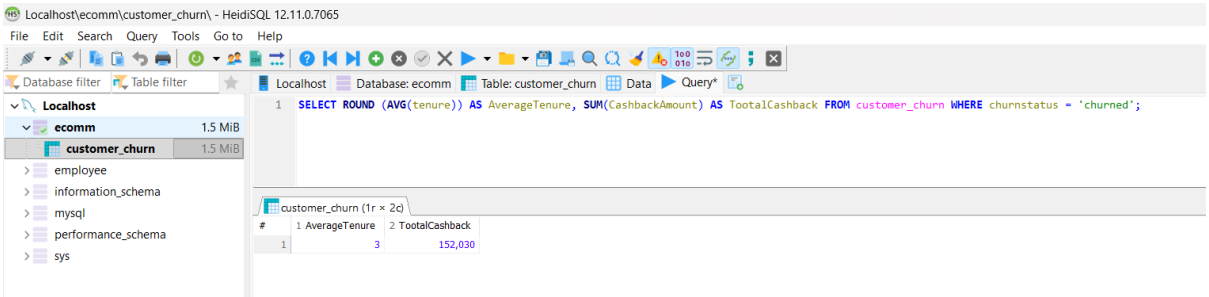
ALTER TABLE customer_churn DROP COLUMN Churn, DROP COLUMN Complain;

Data Exploration and Analysis :

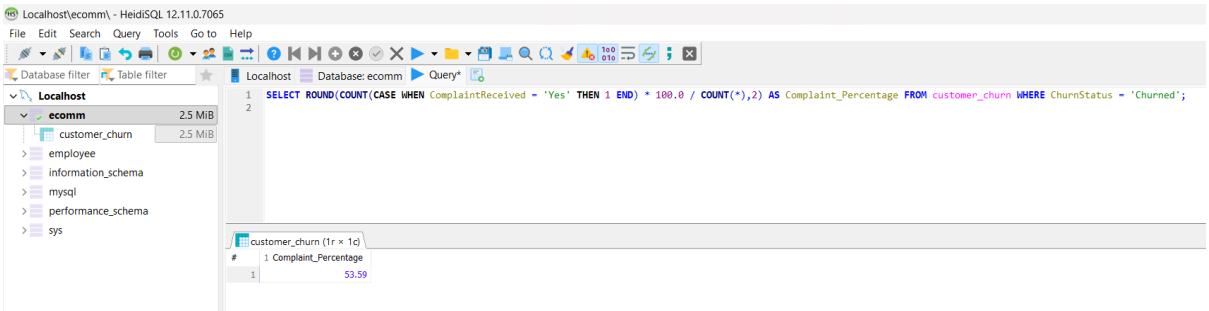
1. SELECT churnStatus, COUNT(*) AS CustomerCount FROM customer_churn GROUP BY churnstatus;



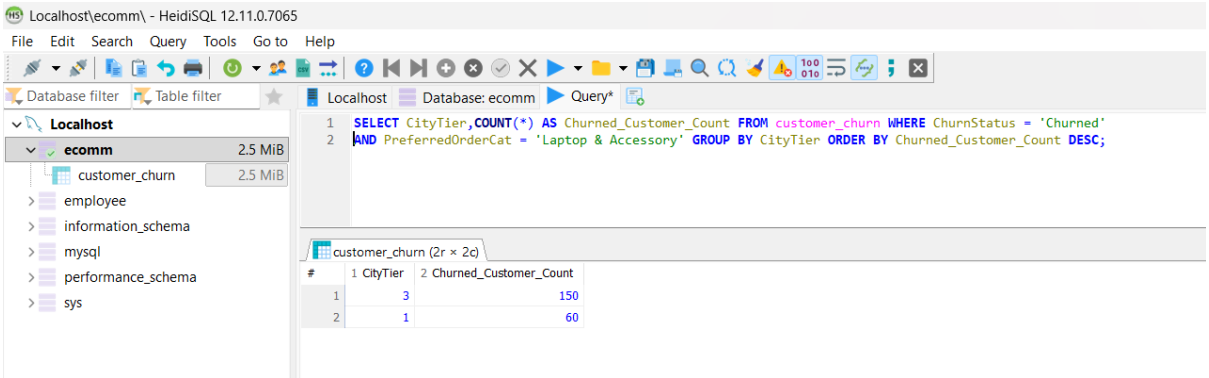
2. SELECT ROUND (AVG(tenure)) AS AverageTenure, SUM(CashbackAmount) AS TootalCashback FROM customer_churn WHERE churnstatus = 'churned';



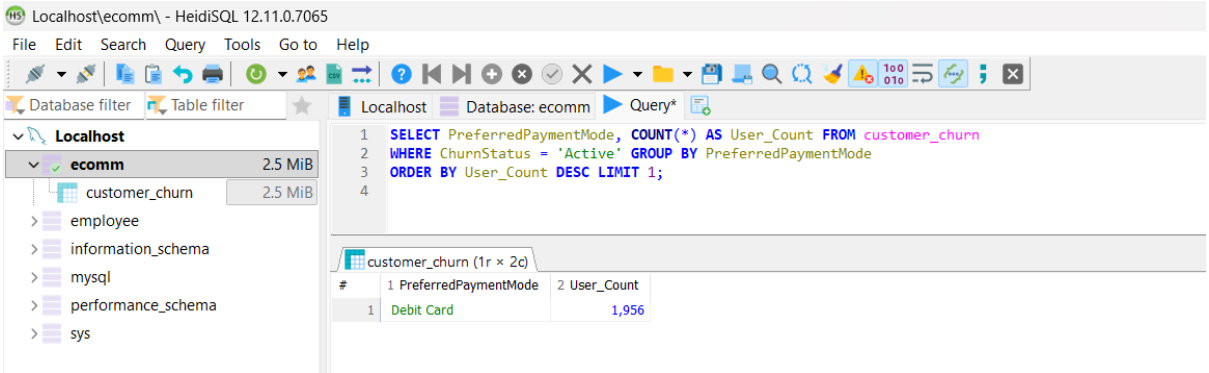
3. SELECT ROUND(COUNT(CASE WHEN ComplaintReceived = 'Yes' THEN 1 END) * 100.0 / COUNT(*),2) AS Complaint_Percentage FROM customer_churn WHERE ChurnStatus = 'Churned';



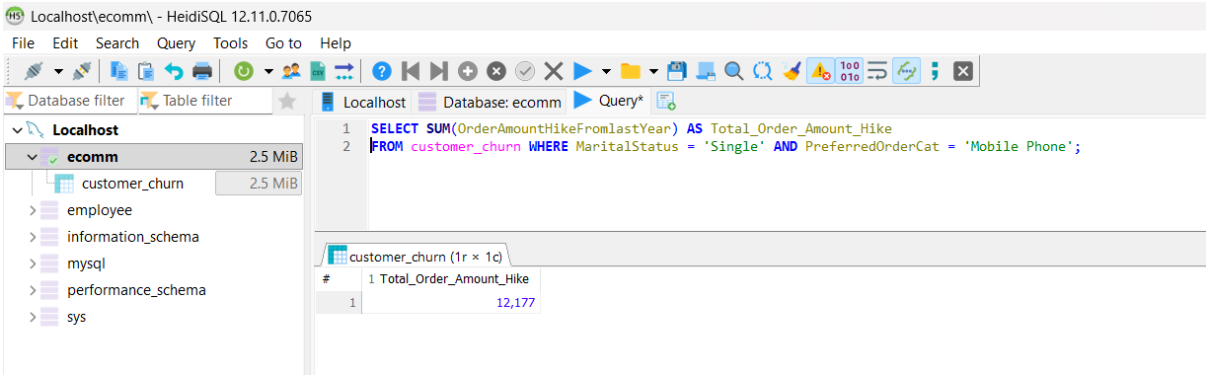
4. SELECT CityTier,COUNT(*) AS Churned_Customer_Count FROM customer_churn WHERE ChurnStatus = 'Churned' AND PreferredOrderCat = 'Laptop & Accessory' GROUP BY CityTier ORDER BY Churned_Customer_Count DESC;



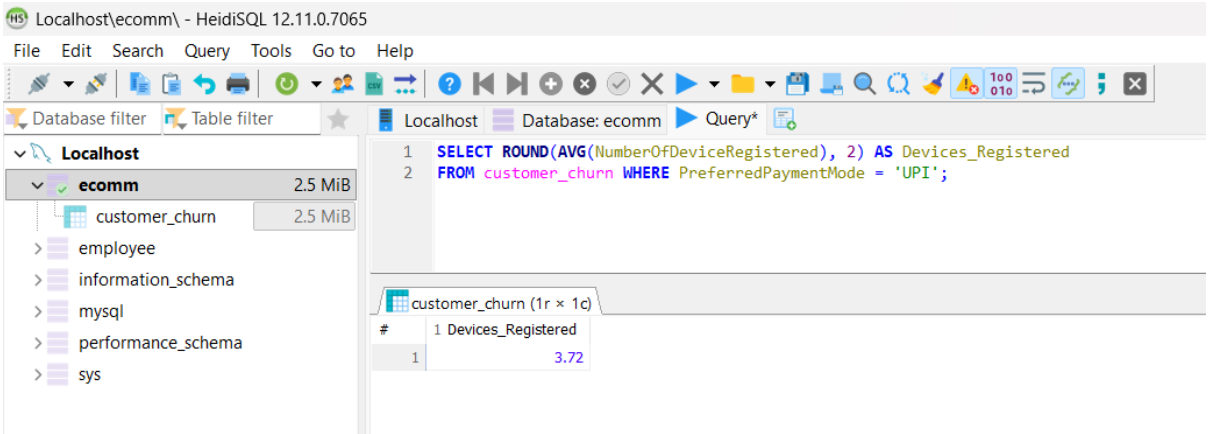
5. SELECT PreferredPaymentMode, COUNT(*) AS User_Count FROM customer_churn WHERE ChurnStatus = 'Active' GROUP BY PreferredPaymentMode ORDER BY User_Count DESC LIMIT 1;



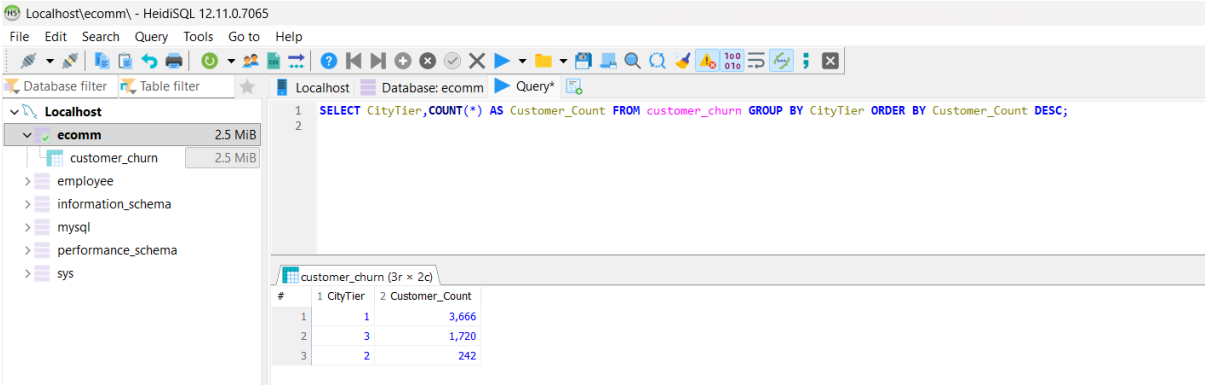
6. SELECT SUM(OrderAmountHikeFromlastYear) AS Total_Order_Amount_Hike FROM customer_churn WHERE MaritalStatus = 'Single' AND PreferredOrderCat = 'Mobile Phone';



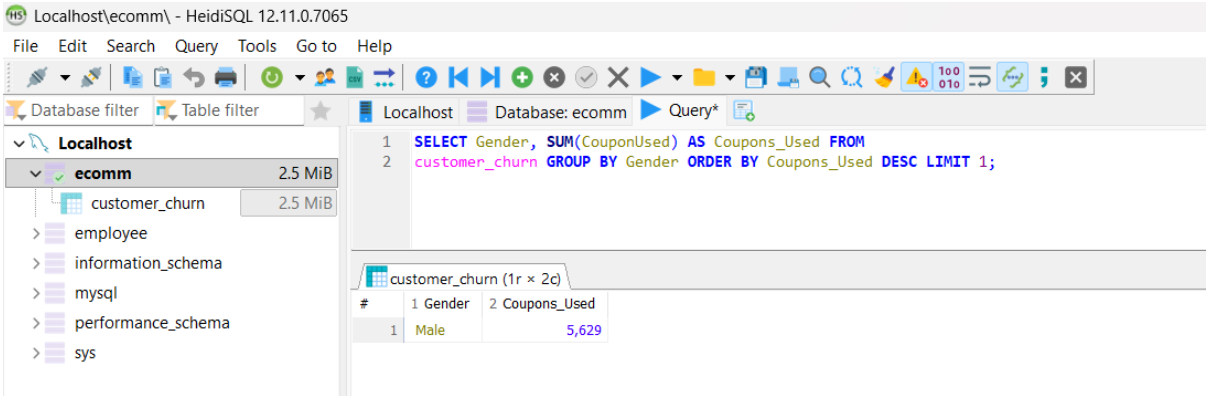
7. SELECT ROUND(AVG(NumberOfDeviceRegistered), 2) AS Devices_Registered FROM customer_churn WHERE PreferredPaymentMode = 'UPI';



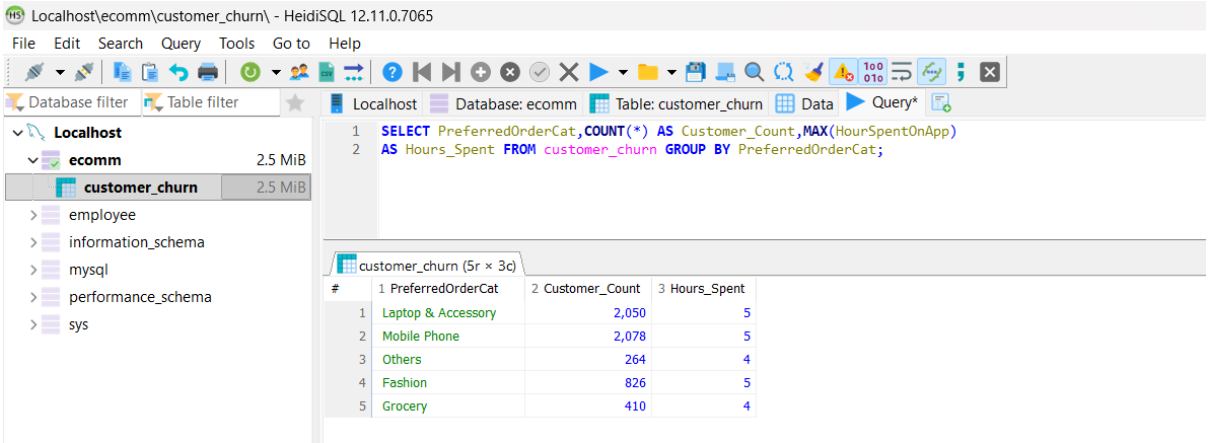
8. SELECT CityTier,COUNT(*) AS Customer_Count FROM customer_churn GROUP BY CityTier ORDER BY Customer_Count DESC;



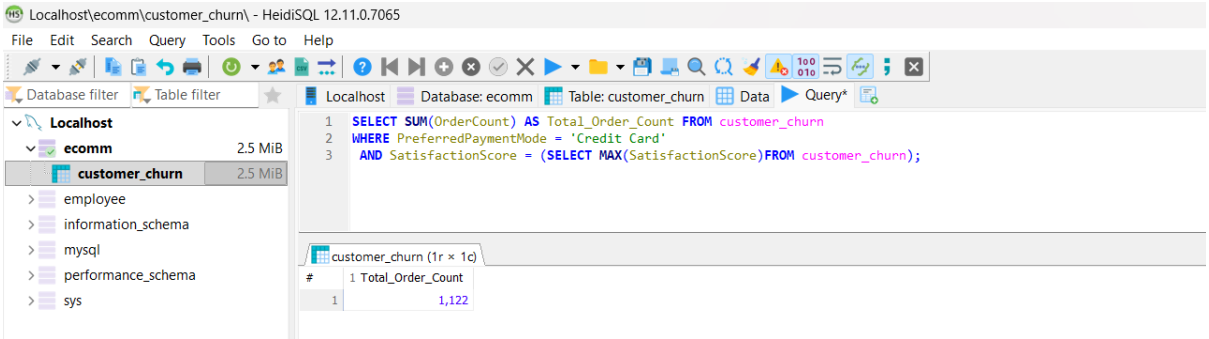
9. SELECT Gender, SUM(CouponUsed) AS Coupons_Used FROM customer_churn GROUP BY Gender ORDER BY Coupons_Used DESC LIMIT 1;



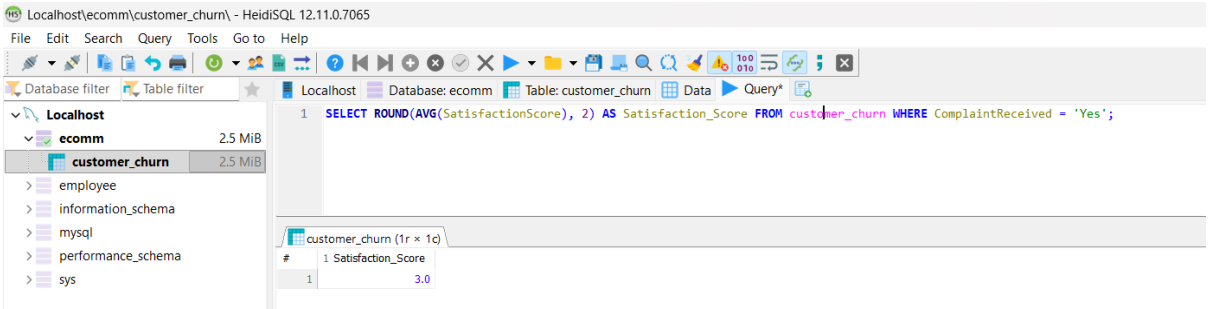
10. SELECT PreferredOrderCat,COUNT(*) AS Customer_Count,MAX(HourSpentOnApp) AS Hours_Spent FROM customer_churn GROUP BY PreferredOrderCat;



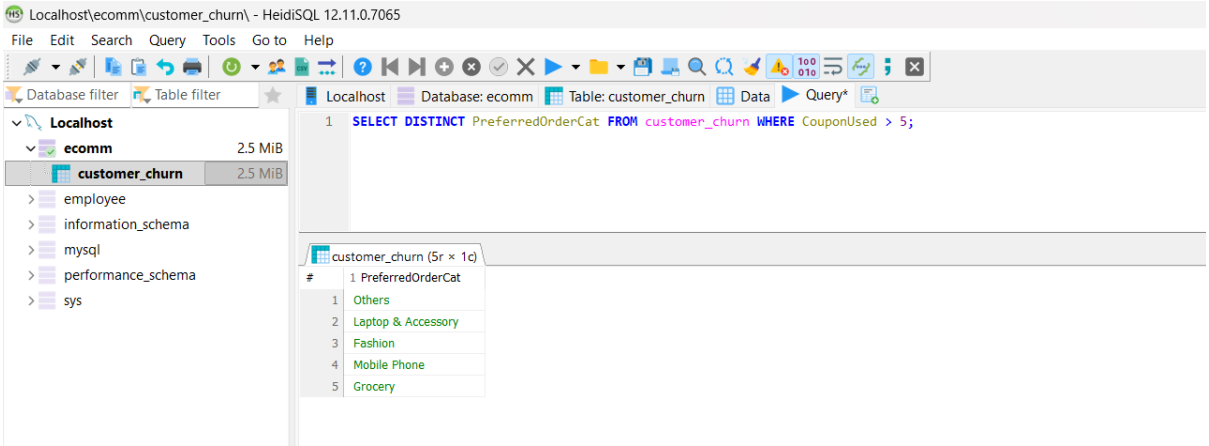
11. SELECT SUM(OrderCount) AS Total_Order_Count FROM customer_churn WHERE PreferredPaymentMode = 'Credit Card' AND SatisfactionScore = (SELECT MAX(SatisfactionScore)FROM customer_churn);



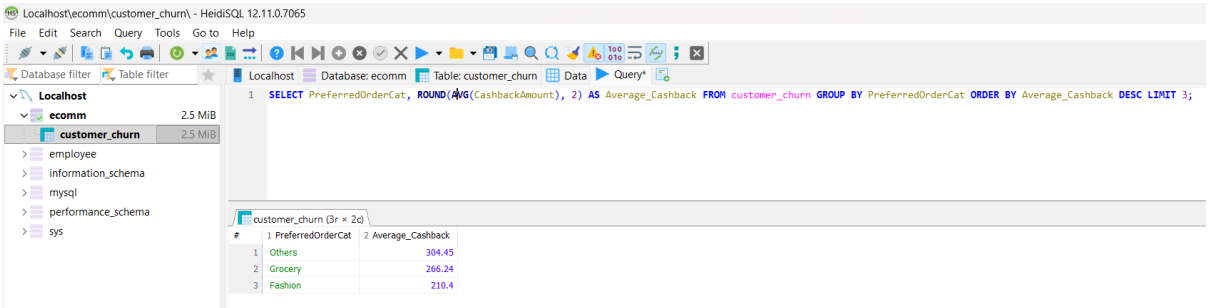
12. SELECT ROUND(AVG(SatisfactionScore), 2) AS Satisfaction_Score FROM customer_churn WHERE ComplaintReceived = 'Yes';



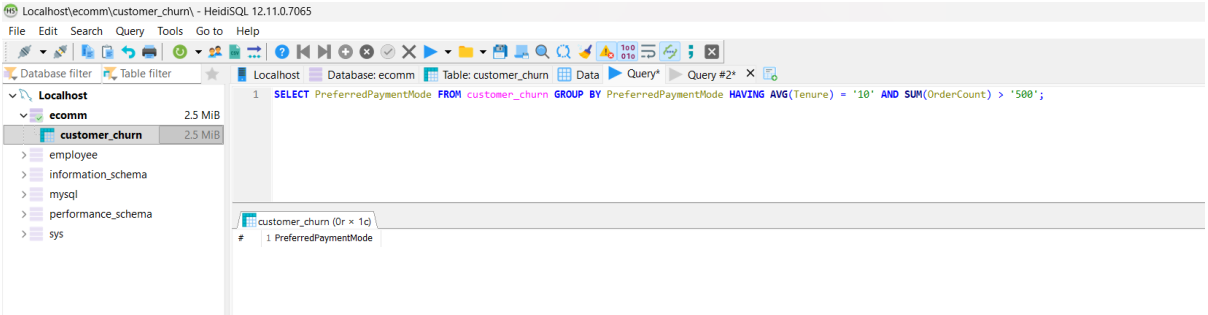
13. SELECT DISTINCT PreferredOrderCat FROM customer_churn WHERE CouponUsed > 5;



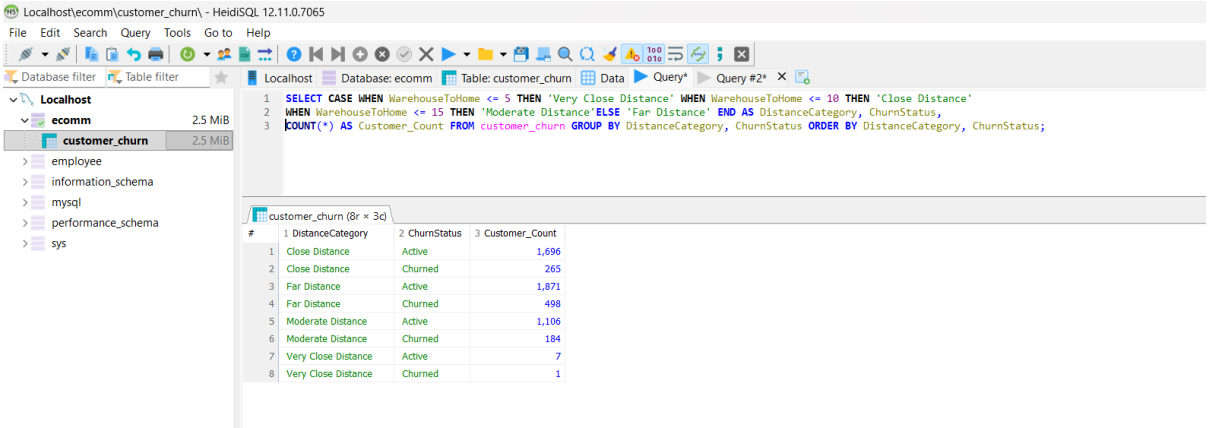
14. SELECT PreferredOrderCat, ROUND(AVG(CashbackAmount), 2) AS Average_Cashback FROM customer_churn GROUP BY PreferredOrderCat ORDER BY Average_Cashback DESC LIMIT 3;



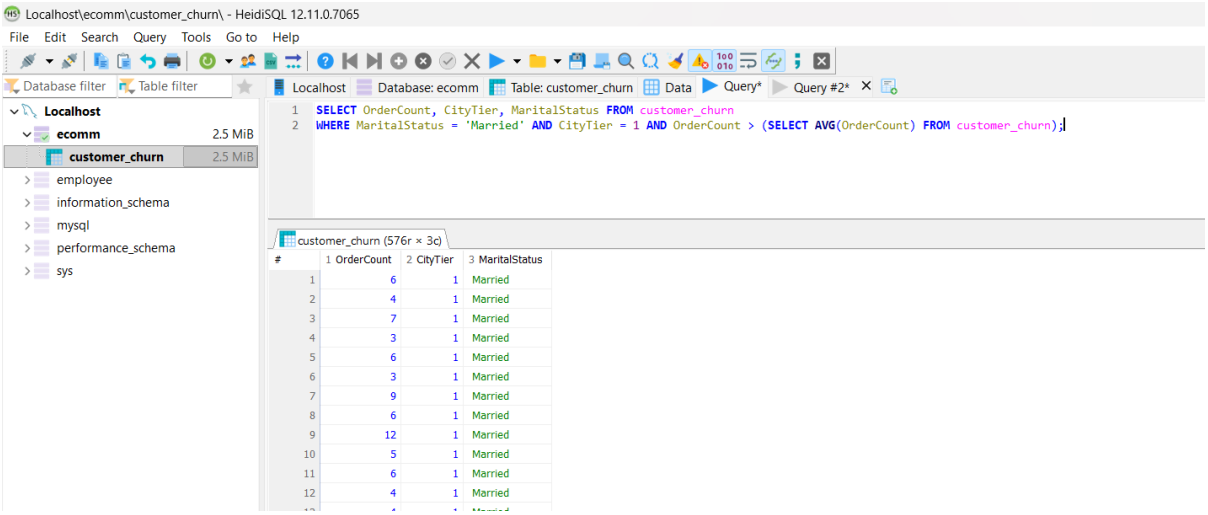
15. SELECT PreferredPaymentMode FROM customer_churn GROUP BY PreferredPaymentMode HAVING AVG(Tenure) = '10' AND SUM(OrderCount) > '500';



16. SELECT CASE WHEN WarehouseToHome <= 5 THEN 'Very Close Distance' WHEN WarehouseToHome <= 10 THEN 'Close Distance' WHEN WarehouseToHome <= 15 THEN 'Moderate Distance'ELSE 'Far Distance' END AS DistanceCategory, ChurnStatus,COUNT(*) AS Customer_Count FROM customer_churn GROUP BY DistanceCategory, ChurnStatus ORDER BY DistanceCategory, ChurnStatus;

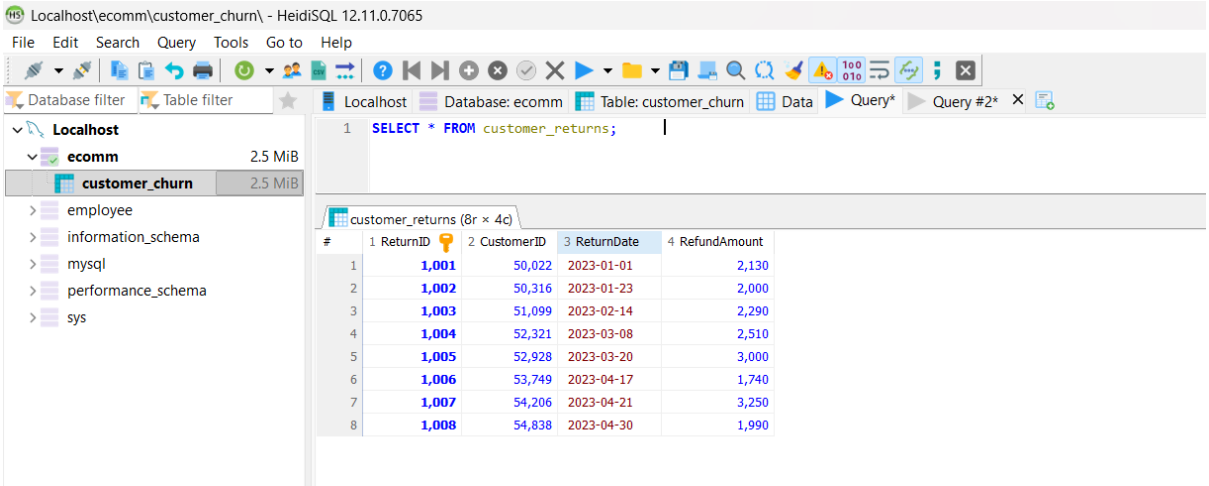


17. SELECT OrderCount, CityTier, MaritalStatus FROM customer_churn WHERE MaritalStatus = 'Married' AND CityTier = 1 AND OrderCount > (SELECT AVG(OrderCount) FROM customer_churn);



18. CREATE TABLE customer_returns (ReturnID INT PRIMARY KEY,CustomerID INT,ReturnDate DATE,RefundAmount INT);

    INSERT INTO customer_returns (ReturnID, CustomerID, ReturnDate, RefundAmount) VALUES (1001, 50022, '2023-01-01', 2130),(1002, 50316, '2023-01-23', 2000),(1003, 51099, '2023-02-14', 2290),(1004, 52321, '2023-03-08', 2510),(1005, 52928, '2023-03-20', 3000),(1006, 53749, '2023-04-17', 1740),(1007, 54206, '2023-04-21', 3250),(1008, 54838, '2023-04-30', 1990);



SELECT c.CustomerID, c.ChurnStatus, c.ComplaintReceived, r.ReturnID, r.ReturnDate, r.RefundAmount FROM customer_churn c INNER JOIN customer_returns r ON c.CustomerID = r.CustomerID WHERE c.ChurnStatus = 'Churned' AND c.ComplaintReceived = 'Yes';

Localhost\ecomm\customer_churn\ - HeidiSQL 12.11.0.7065

File Edit Search Query Tools Go to Help

Database filter | Table filter

- Localhost
  - ecomm 2.5 MiB
    - customer_churn 2.5 MiB
  - employee
  - information_schema
  - mysql
  - performance_schema
  - sys

Localhost | Database: ecomm | Table: customer_churn | Data | Query* | Query #2*

```
1 SELECT c.CustomerID, c.ChurnStatus, c.ComplaintReceived, r.ReturnID, r.ReturnDate, r.RefundAmount
2 FROM customer_churn c INNER JOIN customer_returns r ON c.CustomerID = r.CustomerID
3 WHERE c.ChurnStatus = 'Churned' AND c.ComplaintReceived = 'Yes';
```

customer_churn (3r × 6c)

#	1 CustomerID	2 ChurnStatus	3 ComplaintReceived	4 ReturnID	5 ReturnDate	6 RefundAmount
1	50,316	Churned	Yes	1,002	2023-01-23	2,000
2	52,321	Churned	Yes	1,004	2023-03-08	2,510
3	53,749	Churned	Yes	1,006	2023-04-17	1,740